

1. Message formats

Client → Server : Connection start <client_id> <file_name>

Server → Client : received <saved_bytes>

- If saved_bytes == 0, 새로 업로드 (starting a fresh upload.)
- If saved_bytes > 0, 이전 업로드가 중단되었기에 client 해당 offset부터 재전송 (resuming from saved_bytes offset.)

Client -> Server : Sending Data <read_cnt>

: Client는 파일에서 saved_bytes 위치로 이동한 후 chunk 단위로 보낸다.

- If <read_cnt> == 0, 업로드할 데이터가 더 이상 없으므로 break.
- If <read_cnt> < 0, fread 오류이므로 break.
- Else, send a data chunk to server (buf 내용을 그대로 서버로 전송한다)

Server → Client : Confirming saved bytes to client: <saved_bytes>

(The DATA / Confirming exchange repeats until the entire file is transmitted.)

Server → Client FIN : <final_bytes>

[In terminal, Server Side]

blue - not shown in the terminal, only logical

black - shown in the terminal

Connection established with client <client_id> <file_name>

Sending <saved_bytes> to client

- inform client of previously saved_bytes
- if saved_bytes > 0, the server will expect to the client to resume from that position

[<client_id>] Confirming saved bytes to client: <saved_bytes>

... (Repeated while receiving file chunks)

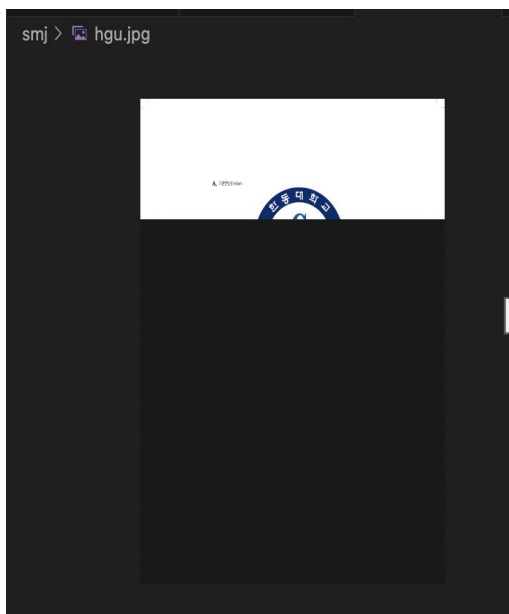
Sending FIN <final_bytes> to client

when <saved_bytes> == 0,

```
root@2c68fa94f8d7:~# ./hw3_server 9190
Connection established with client mj hgu.jpg
Sending 0 bytes in mj/hgu.jpg to client
[mj] Confirming saved bytes to client: 1024
[mj] Confirming saved bytes to client: 2048
[mj] Confirming saved bytes to client: 3072
[mj] Confirming saved bytes to client: 4096
[mj] Confirming saved bytes to client: 5120
[mj] Confirming saved bytes to client: 6144
[mj] Confirming saved bytes to client: 7168
[mj] Confirming saved bytes to client: 8192
[mj] Confirming saved bytes to client: 9216
```

when <saved_bytes> > 0,

```
[smj] Confirming saved bytes to client: 21504
Sending FIN 21504 bytes to client
Connection established with client smj hgu.jpg
Sending 21504 bytes in smj/hgu.jpg to client
[smj] Confirming saved bytes to client: 22528
[smj] Confirming saved bytes to client: 23552
[smj] Confirming saved bytes to client: 24576
[smj] Confirming saved bytes to client: 25600
[smj] Confirming saved bytes to client: 26624
[smj] Confirming saved bytes to client: 27648
[smj] Confirming saved bytes to client: 28672
[smj] Confirming saved bytes to client: 29696
```



[In terminal, Client Side]

blue - not shown in the terminal, only logical

black - shown in the terminal

Connection start <client_id> <file_name>

Received <saved_bytes> from server

→ If saved_bytes == 0, starting a fresh upload and start from the beginning

→ If saved_bytes > 0, resuming from saved_bytes offset from the saved_bytes offset

If saved_bytes == file_size, <client_id>/<file_name> is already fully saved on the server;
close the connection and return 0

When sending data <read_cnt>:

- If <read_cnt> <= 0, break
- Else, send the chunk data to the server (not shown in the terminal, only logical)

Server confirms <saved_bytes> saved

... (Repeated until all chunks are sent)

Received FIN <final_bytes> from server

close connection and return 0

when <saved_bytes> > 0 and <saved_bytes> == <file_size>,

```
root@84dedf870789:~# ./hw3_client 172.17.0.2 9190 myeongjin4 hgu2.jpg
Connection start myeongjin4 hgu2.jpg
Received 307171 from server
resuming from saved_bytes offset
myeongjin4/hgu2.jpg is already saved in server
```

when <saved_bytes> > 0, 파일 끝인 구간부터 재전송

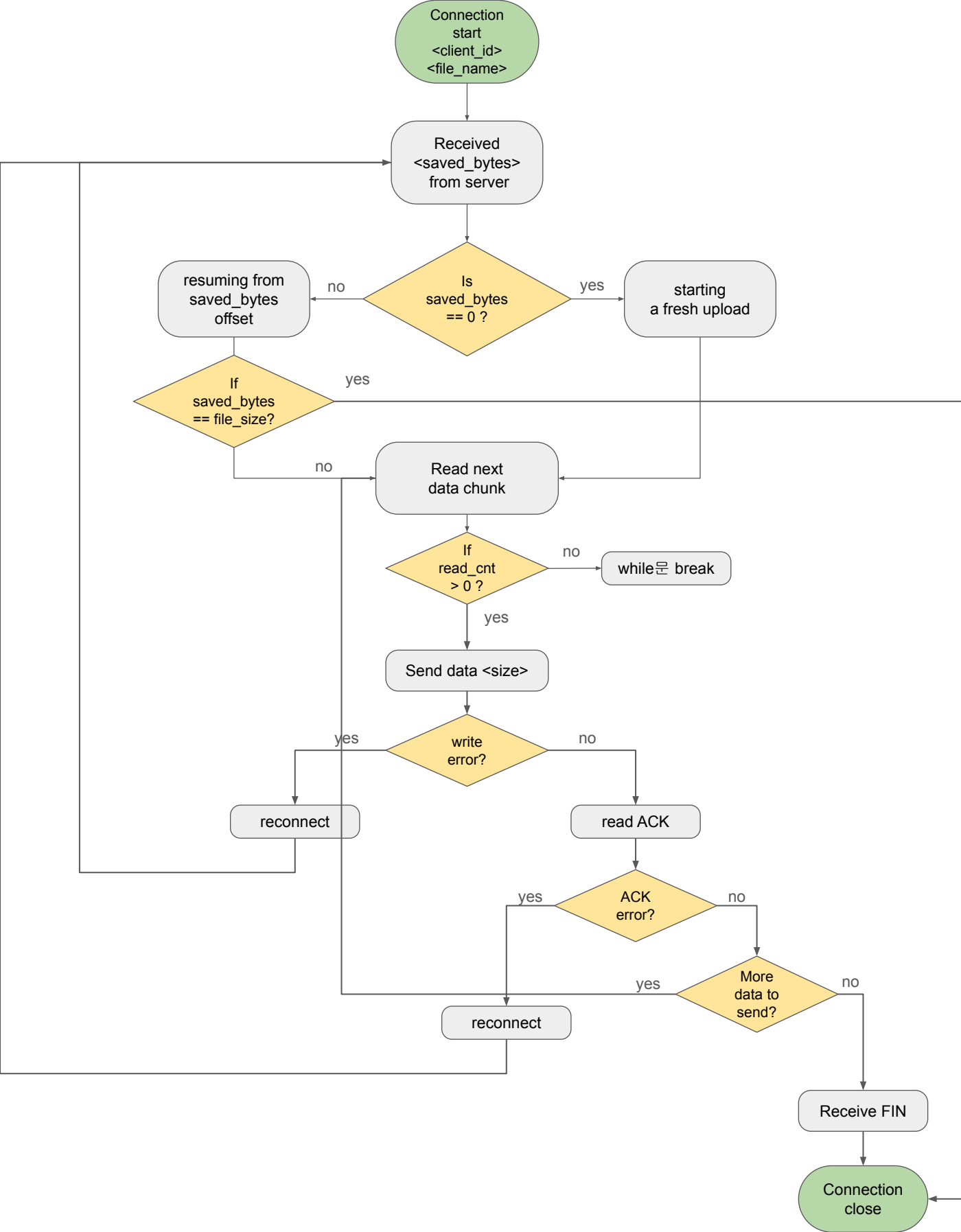
```
Server confirms 7168 saved
Server confirms 8192 saved
^C
root@84dedf870789:~# gcc hw3_client.c -o hw3_client
root@84dedf870789:~# ./hw3_client 172.17.0.2 9190 chloe hgu3.jpg
Connection start chloe hgu3.jpg
Received 8192 from server
resuming from saved_bytes offset
Server confirms 9216 saved
```

when <saved_bytes> == 0,

```
root@84dedf870789:~# ./hw3_client 172.17.0.2 9190 myeongjin6 hgu2.jpg
Connection start myeongjin6 hgu2.jpg
Received 0 from server
starting a fresh upload
Server confirms 1024 saved
Server confirms 2048 saved
Server confirms 3072 saved
Server confirms 4096 saved
```

```
Server confirms 302080 saved
Server confirms 303104 saved
Server confirms 304128 saved
Server confirms 305152 saved
Server confirms 306176 saved
Server confirms 307171 saved
end
Received FIN 307171 from server
```

2. Flow chart



3. Error handling

1. write()/read() or send()/recv() failure 처리

: 데이터 전송 중 write() 또는 ACK 수신 중 read()가 -1을 반환(오류) 하면 발생.

- write(sd, buf) 실패 → perror("Disconnected during write().\n"); break;
→ 현재 inner loop 종료
- read(sd, &ack_bytes) 실패 → perror("Disconnected waiting server confirming.\n"); break;
→ inner loop 종료
- inner loop가 끝나면 while(saved_bytes < file_size) 루프의 다음 iteration에서 reconnect_to_server()가 자동 실행됨

2. peer closed connection (read() == 0) 처리

: 서버가 정상 종료 또는 비정상 종료하면 소켓이 닫혀서 read()가 0을 반환.

- 데이터 전송 중 서버가 연결을 끊으면
→ read(sd, &ack_bytes) 가 0을 반환 → inner loop break
→ 다음 iteration에서 reconnect_to_server() 호출
→ resume 진행

3. Handling Connection Reset

: 네트워크 reset 또는 서버의 abrupt close

- read() or write()가 음수 → break → 다음 iteration에서 reconnect → resume.

4. Handling Server Crash & Restart

: 서버 프로세스가 죽고 다시 실행됨.

- 클라이언트는 서버가 보내준 saved_bytes만 신뢰함
- 서버가 crash 후 restart되면
→ 서버는 기존 파일 크기(saved_bytes)를 다시 알려줌. 클라이언트는 그 offset부터 다시 업로드

5. Handling Client Crash & Restart

: 클라이언트가 중간에 죽었다가 다시 실행됨.

- 클라이언트는 START(client_id, file_name)를 보냄
- 서버는 기존 saved_bytes를 알려줌
- 클라이언트는 파일 포인터를 saved_bytes로 이동
- resume

6. Handling Multiple Interruptions During Upload

업로드 중 아무 위치에서라도 서버나 클라이언트가 죽거나 ACK loss등이 발생해도 현재까지 저장된 바이트(saved_bytes)를 기준으로 클라이언트는 어디까지 업로드되었는지 다시 계산할 필요 없이 이 값만 받아서 resume할 수 있음.